
Volatility Forecasting Shootout

A Comparative Study of GARCH, XGBoost, and LSTM Models
for One-Day-Ahead Realized Volatility Forecasting on the S&P 500

Amit Pareek

M.Stat

Quantitative Research Placement Preparation

July 2026

Abstract

We conduct a controlled comparison of three volatility-forecasting approaches: a classical GARCH(1,1) econometric model, a gradient-boosted tree ensemble (XGBoost), and a recurrent neural network (LSTM), on one-day-ahead realized-variance forecasting for the S&P 500 index ($\tilde{\text{GSPC}}$) over 2012–2026. All three models are evaluated under an identical, leakage-checked feature set and a walk-forward (expanding-window) backtesting protocol, and scored using RMSE, MAE, and QLIKE, with statistical significance assessed via the Diebold–Mariano test. Pooled over the full test period, the LSTM achieves the lowest RMSE and MAE, and this advantage is statistically significant against GARCH ($p = 0.012$). However, QLIKE, the scoring rule designed to be robust to a noisy volatility proxy and to penalize under-prediction of large moves, ranks GARCH(1,1) first. A regime-conditional breakdown resolves the apparent contradiction: XGBoost and the LSTM dominate on Low- and Medium-volatility days, while GARCH(1,1) is the best model, on both RMSE and QLIKE, specifically within the High-volatility tercile of the test set. All three models under-predict the magnitude of the largest volatility spikes in the sample by roughly 40–55%. We argue that this regime-dependent reversal, rather than the pooled leaderboard, is the more decision-relevant finding for a production deployment, and we conclude with a discussion of the deployment trade-offs, explicit assumptions, limitations, and directions for extending the work.

Contents

Abstract	1
1 Introduction	3
1.1 Background	3
1.2 Objective and Scope	3
2 Data and Feature Engineering	5
2.1 Data Source and Instrument	5

2.2	Sample Period and Dataset Size	5
2.3	Preprocessing	5
2.4	Target Variable	6
2.5	Causal Feature Alignment	6
2.6	Feature Set	7
3	Backtesting Methodology	8
3.1	Chronological Split	8
3.2	Walk-Forward Re-estimation	8
4	Models	9
4.1	GARCH(1,1)	9
4.2	XGBoost	9
4.3	LSTM	9
5	Evaluation Metrics	11
5.1	RMSE and MAE	11
5.2	QLIKE	11
5.3	Diebold–Mariano Test	11
6	Results	13
6.1	Pooled Performance	13
6.2	Forecast Visualizations	14
6.3	Calibration Analysis	17
6.4	Regime-Conditional Performance	18
6.4.1	Setup	18
6.4.2	Results	18
6.4.3	Observations	19
6.5	Tail-Spike Capture	19
6.5.1	Setup	19
6.5.2	Results	19
6.5.3	Observations	19
7	Discussion	20
7.1	Deployment Recommendation	20
8	Assumptions and Limitations	22
9	Conclusion	23
A	Full Implementation Code	24

Chapter 1

Introduction

1.1 Background

Quantitative research roles increasingly require the ability to move fluidly between classical statistical models and modern machine-learning methods, to evaluate them fairly on the same problem, and to recommend a specific approach for deployment rather than simply reporting whichever model achieves the lowest error metric. This project was undertaken to build hands-on experience with exactly that workflow, ahead of interviews for a quantitative research placement whose role description explicitly asks candidates to identify a shortlist of potential modelling approaches, evaluate their effectiveness on large historical datasets, and propose the most suitable solution for implementation, spanning classical statistical methods to cutting-edge deep learning techniques.

Volatility forecasting was chosen as the underlying problem for three reasons. First, it is a canonical, well-studied quantitative finance problem with a long econometric literature, making it possible to sanity-check results against known stylized facts such as volatility clustering, the leverage effect, and fat tails. Second, it connects directly to material being concurrently studied in Hull’s *Options, Futures, and Other Derivatives*: volatility is the central input to option pricing, and the RiskMetrics-style exponentially weighted moving average (EWMA) estimator used as one of the features below is drawn directly from Hull’s treatment of Value-at-Risk. Third, it admits a genuinely fair three-way comparison: a classical econometric model (GARCH), a classical machine-learning model (gradient-boosted trees), and a deep-learning model (LSTM) can all be framed as solving the identical one-day-ahead prediction task from the same information set, which is not true of every finance problem.

1.2 Objective and Scope

The objective is to build, evaluate, and fairly compare the three models described above under a walk-forward backtesting protocol, to diagnose *where* each model wins or fails

(not merely whether it wins on a single pooled metric), and to arrive at an explicit, defensible deployment recommendation. The scope is intentionally limited to daily-frequency data on a single, liquid index, so that methodological correctness (leakage prevention, proper scoring rules, regime-conditional evaluation) could be prioritized over engineering effort. Section ?? states the resulting limitations explicitly.

The remainder of this report proceeds as follows. Chapter 2 describes data collection and feature engineering. Chapter 3 describes the backtesting methodology. Chapter 4 details the three models. Chapter 5 defines the evaluation metrics. Chapter 6 presents results. Chapter 7 discusses the findings. Chapter 8 states assumptions and limitations. Chapter 9 concludes. Appendix A reproduces the complete implementation.

Chapter 2

Data and Feature Engineering

2.1 Data Source and Instrument

Daily price data was collected programmatically via the `yfinance` Python library, which queries Yahoo Finance’s public historical data endpoint. The instrument used is the S&P 500 index under its Yahoo Finance ticker `^GSPC`, sampled at daily frequency (one observation per trading day, adjusted close).

The S&P 500 was chosen, rather than a single equity or a less liquid index, for two reasons: it is the instrument most frequently used in the published academic literature on GARCH/ML/DL volatility comparisons, allowing rough sanity-checking of the findings below against known results; and its long, continuously-traded history minimizes missing-data and corporate-action complications relative to a single stock.

2.2 Sample Period and Dataset Size

The sample spans 2012-01-01 to 2026-07-01, yielding 3,621 usable daily observations after feature construction removes the initial rows consumed by lagged and rolling-window features. This window was deliberately kept modest in size: a few thousand rows, a few hundred kilobytes as a CSV. The objective was to master correct methodology (leakage prevention, walk-forward validation, proper scoring rules) rather than to engineer around large-data infrastructure concerns. The window still includes at least one genuine volatility regime shift (the COVID-19 shock and its aftermath), which is sufficient to stress-test regime-dependent model behaviour without requiring pre-2008 data whose market microstructure differs substantially from the present day.

2.3 Preprocessing

From the raw daily closing price series P_t , log returns are computed as

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right), \quad (2.1)$$

and rows with missing data (the first row, where P_{t-1} is undefined) are dropped. No further cleaning was required, since Yahoo Finance’s adjusted-close series already accounts for splits and dividends for index data of this type.

2.4 Target Variable

Definition 2.1 (Realized-variance proxy). *In the absence of intraday data, the standard proxy for realized variance on day t is*

$$\text{target_var}[t] = r_t^2, \quad \text{target_vol}[t] = \sqrt{252 \cdot \text{target_var}[t]}, \quad (2.2)$$

i.e. the squared daily log return, annualized by the usual $\sqrt{252}$ trading-day scaling. This is a noisy but unbiased estimator of the true latent daily variance.

Remark 2.2. *Squared daily returns are a well-known noisy proxy for true realized variance: a single day’s squared return can deviate substantially from the underlying variance process, particularly since intraday price paths are not observed. This noise is precisely why QLIKE (Section 5.2), rather than RMSE alone, is emphasized in the volatility-forecasting literature: QLIKE has been shown to preserve the correct ranking of competing forecasts even when the evaluation target itself is a noisy proxy, whereas squared-error-based rankings can be distorted by that noise.*

2.5 Causal Feature Alignment

Row t represents “standing at the close of day $t - 1$, forecasting day t .” Every feature at row t is therefore constructed using only $r_{t-1}, r_{t-2}, \dots$, never r_t itself, since $\text{target_var}[t]$ is a deterministic function of r_t and any feature leaking r_t would make the model appear artificially accurate in-sample while being useless out-of-sample.

This causal alignment was verified empirically during development. An early version of the feature pipeline inadvertently included r_t in a lag feature, an off-by-one indexing error. It was caught because GARCH, fit directly on the process that generated a synthetic test dataset, unexpectedly failed to outperform the other models on that synthetic data, prompting a review that surfaced the leakage. This episode is retained in the discussion here because reproducing it is one of the more useful checks available when auditing a time-series pipeline: fit the simplest correct model on data simulated from a known process, and treat its failure to win as a signal worth investigating rather than dismissing.

2.6 Feature Set

The complete feature vector supplied to the XGBoost and LSTM models (GARCH instead uses only the raw return series, per its parametric specification) consists of:

- return lags $r_{t-1}, r_{t-2}, \dots, r_{t-5}$ (5 features);
- squared return lags $r_{t-1}^2, r_{t-2}^2, \dots, r_{t-5}^2$ (5 features);
- rolling standard deviation of r over trailing windows of 5, 10, and 21 trading days, computed using only r_{t-1} and earlier (3 features);
- rolling mean absolute return over the same three windows (3 features);
- RiskMetrics-style EWMA volatility, $\hat{\sigma}_t^2 = \lambda \hat{\sigma}_{t-1}^2 + (1 - \lambda)r_{t-1}^2$, $\lambda = 0.94$, annualized (1 feature).

This gives 17 features in total. The EWMA feature is drawn directly from the exponentially-weighted volatility estimator presented in Hull's chapter on Value-at-Risk.

Chapter 3

Backtesting Methodology

3.1 Chronological Split

The 3,621 observations are split chronologically, never shuffled, into a training set of 2,897 observations and a held-out test set of the most recent 724 observations (an 80/20 split). Shuffling was avoided because it would leak future information into the training set, which is a more severe error for time-series data than for i.i.d. data.

3.2 Walk-Forward Re-estimation

A single static fit evaluated once on a hold-out set understates the realistic operational challenge of a deployed volatility model, which must be periodically re-estimated as new data arrives. Walk-forward (expanding-window) validation is therefore used for the two models whose retraining cost is low enough to make it practical:

- **GARCH(1,1)** is re-estimated every 21 trading days (approximately monthly) on an expanding window of all data observed to date. Between full re-estimations, the conditional-variance recursion is still advanced through newly realized returns using the `arch` library’s parameter-fixing functionality, so that forecasts are never simply repeated stale values.
- **XGBoost** is re-estimated every 63 trading days (approximately quarterly) on an expanding window.
- **LSTM** is fit once on the full training window and not walk-forward retrained. This is an explicit simplification, discussed in Section 8, made to keep per-run training cost compatible with a free-tier cloud compute budget.

The refit cadences of 21 and 63 trading days were chosen to approximate monthly and quarterly re-estimation schedules that would be operationally realistic for a small research team; they were not tuned against any performance metric.

Chapter 4

Models

4.1 GARCH(1,1)

The classical baseline is a GARCH(1,1) model, in which the conditional variance of returns follows

$$r_t = \sigma_t \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1), \quad \sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2, \quad (4.1)$$

with parameters (ω, α, β) estimated by maximum likelihood on the (percentage-scaled) return series, using the `arch` Python package. This model directly encodes volatility clustering (via β) and a reaction to the most recent shock (via α), and is the natural statistical baseline given its use of maximum-likelihood estimation and conditional heteroskedasticity.

4.2 XGBoost

The machine-learning entrant is a gradient-boosted tree ensemble (`XGBRegressor` from the `xgboost` library), trained to predict $\sqrt{252 \cdot \text{target_var}[t]}$ directly from the 17-feature vector described in Section 2. Hyperparameters (300 trees, maximum depth 3, learning rate 0.05, 80% row and column subsampling) were set to standard, moderately regularized defaults rather than tuned via cross-validation, since hyperparameter search was judged to be a lower-priority use of project time than getting the backtesting methodology correct.

4.3 LSTM

The deep-learning entrant is a single-layer LSTM network operating on sequences of the last 20 days of the 17-dimensional feature vector, standardized using a scaler fit on

the training set only:

Input $(20 \times 17) \rightarrow \text{LSTM}(32, \tanh) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(1, \text{softplus})$.

The final softplus activation constrains the output to be strictly non-negative, a necessary but easy-to-overlook constraint for volatility forecasts. The network is trained with the Adam optimizer, mean-squared-error loss, a 90/10 train/validation split within the training window, and early stopping (patience of 5 epochs).

Remark 4.1 (Reproducibility and ensembling). *TensorFlow does not guarantee bit-for-bit determinism on CPU even with a fixed random seed, owing to floating-point non-associativity in multi-threaded operations. Across development runs, a single LSTM fit produced pooled RMSE values ranging from 0.1037 to 0.1073 on the identical dataset: not a large difference, but large enough that reporting a single run’s number as the result would be misleading. The results reported in Chapter 6 instead use a 5-seed ensemble average (seeds 0–4), with the seed-to-seed standard deviation reported alongside the point estimate as an explicit measure of residual instability. GARCH and XGBoost, by contrast, are exactly deterministic given the data.*

Chapter 5

Evaluation Metrics

Let y_t denote realized volatility (or variance, as noted) and $\hat{y}_t$ the model forecast, for t in the test set of size n .

5.1 RMSE and MAE

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}, \quad \text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|. \quad (5.1)$$

Both are expressed in volatility units and are straightforward to interpret, but both are dominated by the many ordinary (low-volatility) days in the sample, since squared or absolute error on a calm day is small in absolute terms regardless of which model is used.

5.2 QLIKE

Definition 5.1 (QLIKE loss). *On the variance scale $y_t^{\text{var}} = y_t^2/252$,*

$$\text{QLIKE} = \frac{1}{n} \sum_{t=1}^n \left(\frac{y_t^{\text{var}}}{\hat{y}_t^{\text{var}}} - \ln \frac{y_t^{\text{var}}}{\hat{y}_t^{\text{var}}} - 1 \right). \quad (5.2)$$

QLIKE is the standard scoring rule in the volatility-forecasting literature because, unlike squared-error loss, it remains a valid ranking of forecast quality even when y_t is itself a noisy proxy for the true latent variance, and it asymmetrically penalizes under-prediction of large moves more heavily than over-prediction: precisely the property relevant to risk management.

5.3 Diebold–Mariano Test

To assess whether one model’s apparent edge over another is statistically distinguishable from noise, the Diebold–Mariano test is applied to the squared-error loss differen-

tial $d_t = e_{1,t}^2 - e_{2,t}^2$ between two models' forecast errors:

$$\text{DM} = \frac{\bar{d}}{\sqrt{\widehat{\text{Var}}(d)/n}} \xrightarrow{d} \mathcal{N}(0, 1) \quad \text{under } H_0 : \mathbb{E}[d_t] = 0. \quad (5.3)$$

Chapter 6

Results

6.1 Pooled Performance

Table 6.1 reports each model’s pooled performance over the entire 724-day test period.

Table 6.1: Pooled test-period performance. Bold indicates the best value in each column (lowest, except R^2 where higher is better).

Model	RMSE	MAE	QLIKE	R^2
GARCH(1,1)	0.1141	0.0831	1.6673	−0.0702
XGBoost	0.1122	0.0729	1.9858	−0.0347
LSTM (5-seed ensemble)	0.1064	0.0700	1.8829	0.0700

The mean seed-to-seed standard deviation of the LSTM ensemble’s daily forecasts was 0.0151 (volatility units), against a mean realized volatility level in the test period of order 0.1–0.2: material, but not dominant relative to the forecast errors themselves.

Table 6.2: Diebold–Mariano tests on squared-error loss differentials, pooled test period.

Comparison	DM statistic	p -value	Conclusion (5% level)
GARCH(1,1) vs. XGBoost	0.519	0.604	Not significant
GARCH(1,1) vs. LSTM	2.518	0.012	Significant (LSTM lower loss)
XGBoost vs. LSTM	1.591	0.112	Not significant

Pooled over the full test period, the LSTM ensemble achieves the lowest RMSE, MAE, and the only positive R^2 among the three models, and its advantage over GARCH(1,1) is statistically significant. On QLIKE, however, GARCH(1,1) is the clear winner. This apparent contradiction is investigated in Section 6.4.

6.2 Forecast Visualizations

Figure 6.1 shows the underlying price series and the realized-volatility target series with the train/test split marked. Figures 6.2–6.4 overlay each model’s one-day-ahead forecast against the realized series over the test period.

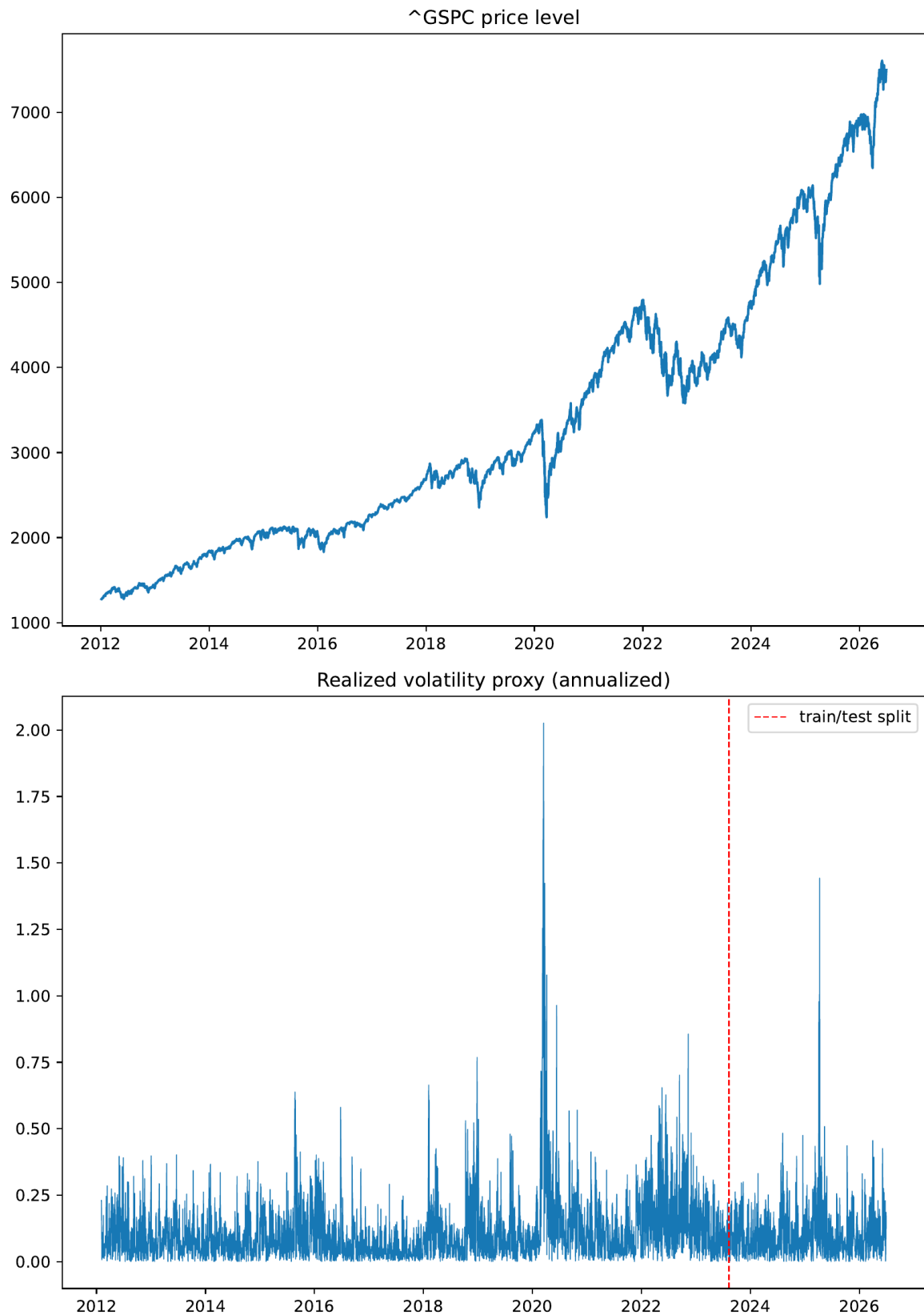


Figure 6.1: Top: S&P 500 ($\hat{GSPC}$) closing price, 2012–2026. Bottom: annualized realized volatility proxy, $\sqrt{252 r_t^2}$, with the train/test split (most recent 20% of observations) marked in red.

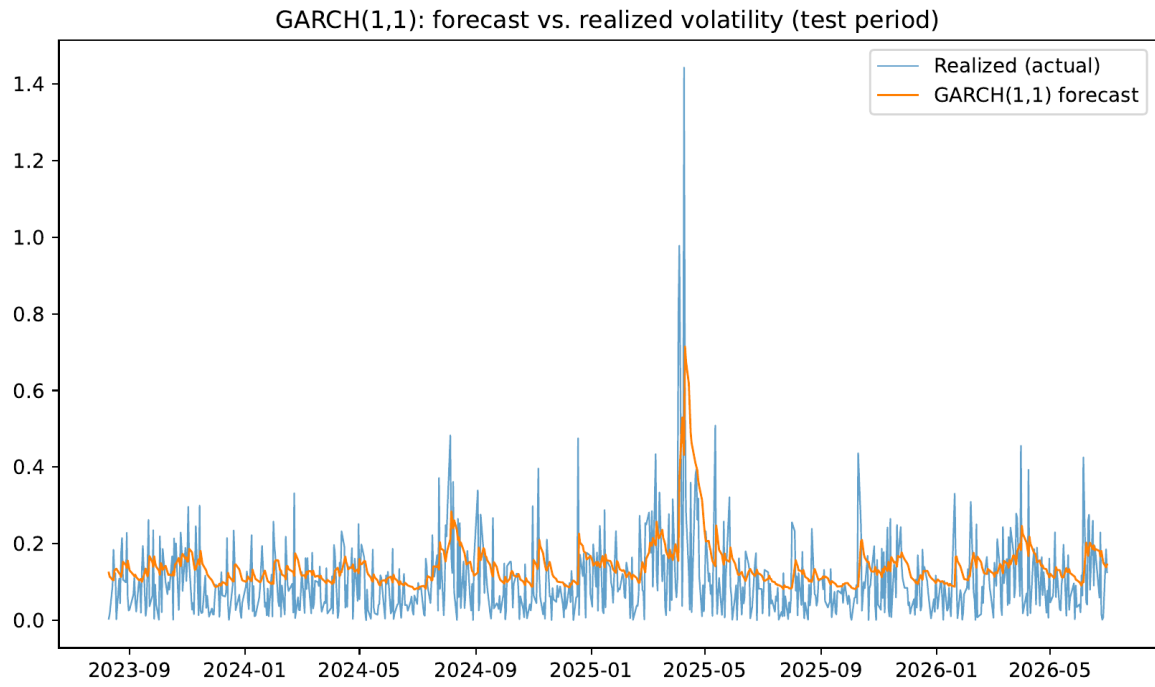


Figure 6.2: GARCH(1,1) one-day-ahead forecast (orange) versus realized volatility (blue), test period.

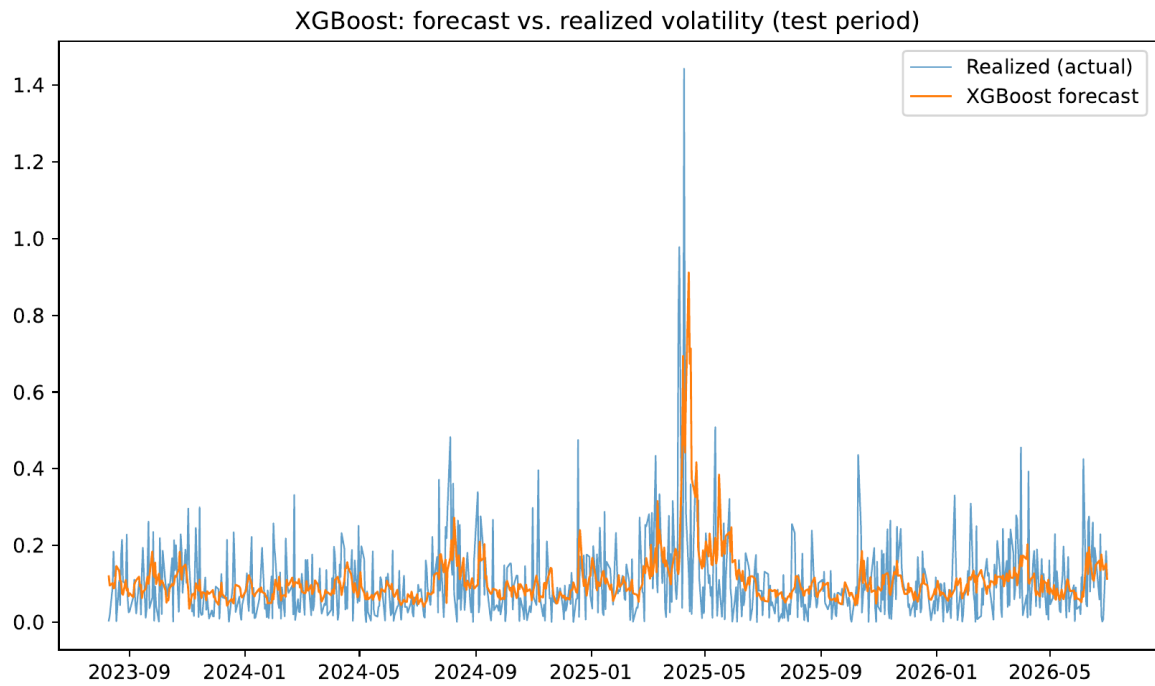


Figure 6.3: XGBoost one-day-ahead forecast (orange) versus realized volatility (blue), test period.

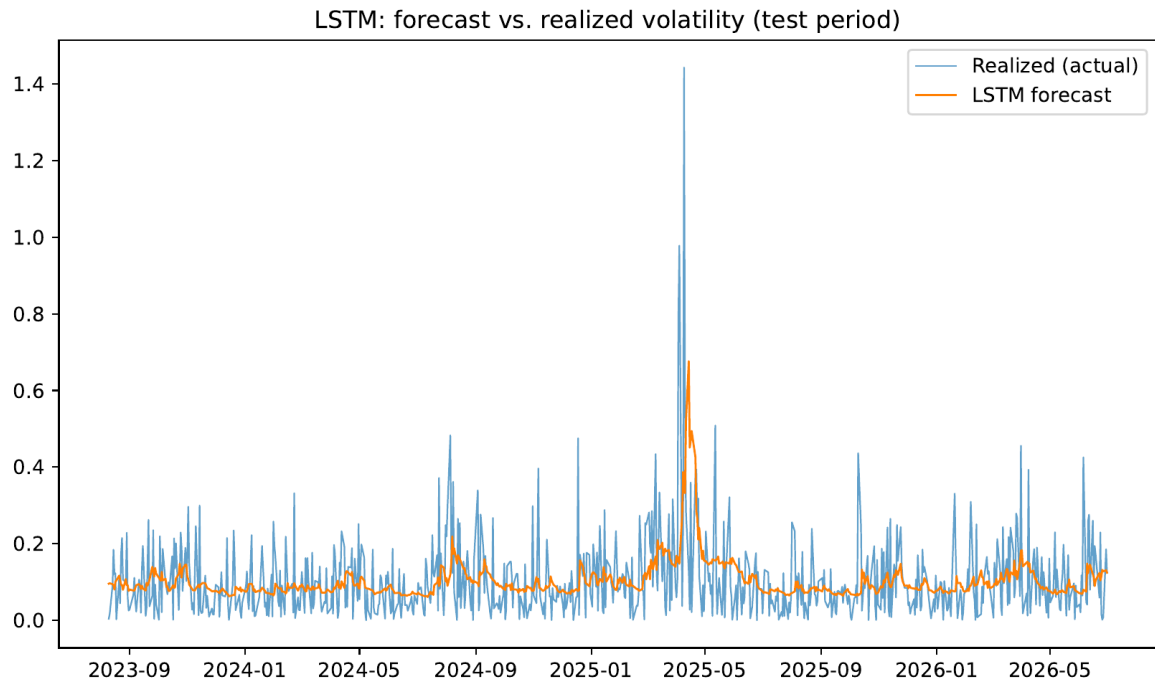


Figure 6.4: LSTM (5-seed ensemble mean) one-day-ahead forecast (orange) versus realized volatility (blue), test period.

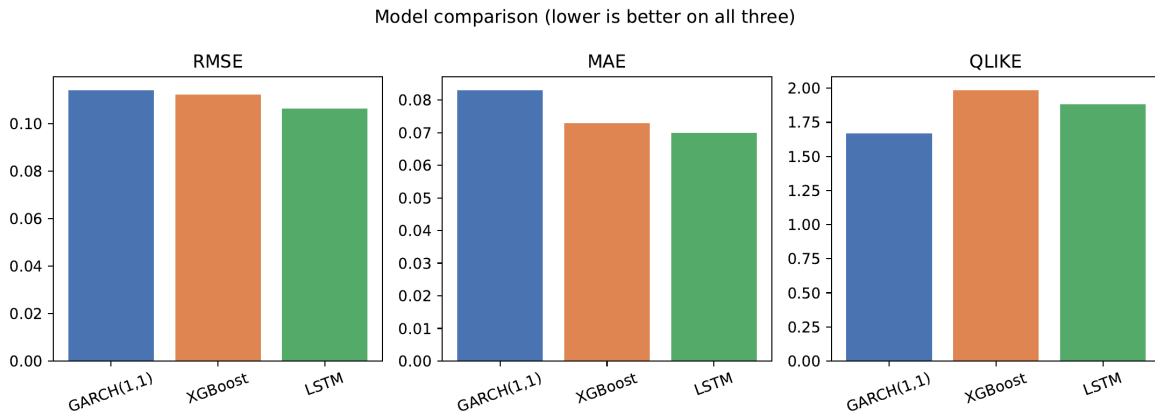


Figure 6.5: Pooled-metric comparison across the three models. Lower is better on all three panels.

6.3 Calibration Analysis

Figure 6.6 plots each model's predicted volatility against the actual realized volatility for every test-period day, with the 45-degree line marking perfect calibration. Points clustering below the line indicate systematic under-prediction.

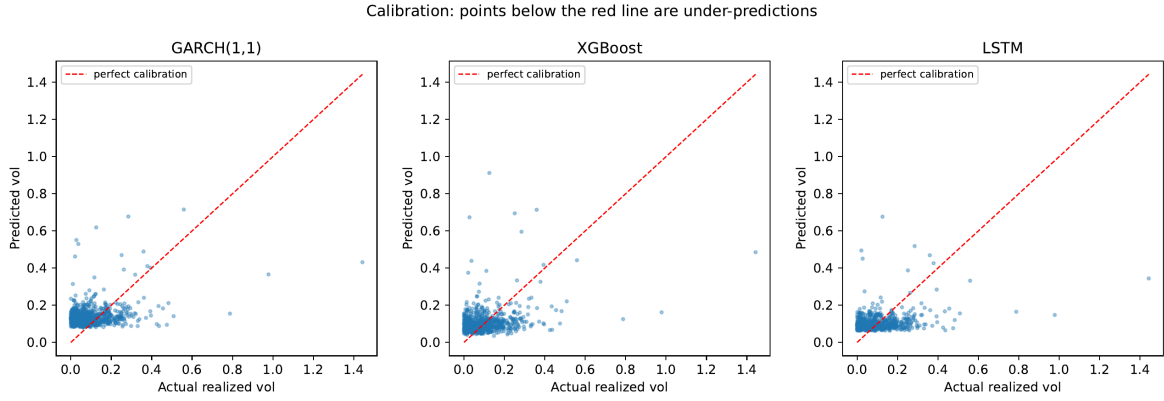


Figure 6.6: Calibration scatter plots. All three models show a visible tendency to under-predict on the highest-volatility days.

6.4 Regime-Conditional Performance

6.4.1 Setup

A single pooled metric can conceal a model that performs well on ordinary days while quietly failing exactly when volatility spikes: the scenario of greatest practical consequence for risk management and options trading. Each model is re-scored separately within Low, Medium, and High realized-volatility terciles of the test set (approximately 241 days each).

6.4.2 Results

Table 6.3: Regime-conditional RMSE and QLIKE. Bold indicates the best model within each regime.

Model	Regime	N	RMSE	QLIKE
GARCH(1,1)	Low	241	0.1255	3.4183
GARCH(1,1)	Medium	241	0.0687	0.4815
GARCH(1,1)	High	242	0.1362	1.1044
XGBoost	Low	241	0.0955	2.7547
XGBoost	Medium	241	0.0531	0.3211
XGBoost	High	242	0.1605	2.8777
LSTM	Low	241	0.0872	2.7853
LSTM	Medium	241	0.0377	0.2212
LSTM	High	242	0.1577	2.6391

6.4.3 Observations

The regime breakdown resolves the pooled-metric contradiction. XGBoost and the LSTM dominate the Low and Medium volatility regimes on both RMSE and QLIKE. GARCH(1,1), despite losing on pooled RMSE, is the best model on both RMSE and QLIKE specifically within the High-volatility regime, by a wide margin on QLIKE (1.10 versus 2.64–2.88 for the other two models). This is a textbook Simpson’s-paradox-shaped result: the pooled ranking and the regime-conditional ranking disagree because the “average day” in the test set is not the day that matters most for risk management.

6.5 Tail-Spike Capture

6.5.1 Setup

For the worst 36 test days by realized volatility (the top 5%), the average ratio of predicted to actual volatility is computed for each model.

6.5.2 Results

Table 6.4: Spike-capture ratio (mean predicted/actual) on the 36 highest-realized-volatility test days.

Model	Predicted / Actual	Interpretation
GARCH(1,1)	0.583	Under-predicts, least severely
XGBoost	0.504	Under-predicts
LSTM	0.455	Under-predicts, most severely

6.5.3 Observations

All three models substantially under-predict the magnitude of the largest volatility spikes in the sample, capturing only 46–58% of the actual move on average. This is a genuine limitation of one-day-ahead forecasting from daily return data alone, not an implementation defect: no model in this comparison observes a shock before it happens. GARCH’s faster reaction to the most recent squared return gives it a real, consistent advantage here relative to the other two models, but the absolute capture rate remains well below 1.0 across the board.

Chapter 7

Discussion

Three findings, taken together, motivate a specific deployment recommendation rather than a single “best model” declaration.

Pooled-metric disagreement. The disagreement between RMSE (LSTM wins) and QLIKE (GARCH wins) is not noise; it reflects the two metrics answering different questions. RMSE is dominated by the many ordinary days in the sample, on which the LSTM’s flexible, smoothed fit is genuinely more accurate. QLIKE weights large-variance days far more heavily, and on those days GARCH’s built-in, immediate reaction to the most recent squared return (the αr_{t-1}^2 term) is a real structural advantage that a model trained to minimize average error over mostly-calm days does not automatically acquire.

Regime concentration of the GARCH advantage. The regime-conditional breakdown confirms this mechanistically: GARCH’s edge is concentrated exactly in the High-volatility tercile, while it is the worst of the three models in the Low and Medium terciles. A production system that only reported the pooled RMSE would have deployed the model that is worse exactly when the cost of being wrong is highest.

Tail-spike under-prediction. All models materially under-predict genuine tail spikes, which tempers any recommendation: none of these models, as built, would be adequate as a sole input to setting risk limits ahead of a shock. This is consistent with the broader econometric understanding that jump risk is not well captured by continuous volatility models estimated on daily data.

7.1 Deployment Recommendation

GARCH(1,1) is recommended as the production default and sanity-check floor, given its determinism, interpretability, cheap re-estimation, and demonstrated advantage during the regime that matters most for risk management. An ML or DL model should only be substituted, or blended in, if it can be shown to improve on GARCH specifically in the Medium/High-volatility regimes without sacrificing GARCH’s responsiveness to fresh shocks. The XGBoost and LSTM models evaluated here do not yet meet that

bar, since their comparative advantage is concentrated in the regime where accuracy is least consequential.

Chapter 8

Assumptions and Limitations

1. **Noisy target proxy.** The realized-variance target (r_t^2) is a noisy, single-day proxy in the absence of intraday data. QLIKE was used specifically to mitigate the resulting ranking distortion, but it does not eliminate the underlying noise.
2. **Gaussian GARCH innovations.** The GARCH model assumes conditionally normal innovations. Financial returns are well known to be fat-tailed; a Student- t or skewed- t specification would likely improve GARCH's tail behaviour further.
3. **Single asset, single frequency.** All results pertain to daily S&P 500 index data only; findings may not generalize to individual equities, other asset classes, or intraday frequencies.
4. **No transaction costs or trading overlay.** This is a pure forecasting-accuracy study; no attempt is made to translate forecasts into a trading or hedging strategy with realistic costs.
5. **LSTM trained once per seed, not walk-forward retrained.** Unlike GARCH and XGBoost, the LSTM is not periodically re-estimated across the test period, for compute-budget reasons. This may understate its achievable performance or overstate it by chance; the 5-seed ensemble mitigates but does not fully resolve this.
6. **Hyperparameters not tuned.** XGBoost and LSTM hyperparameters were set to reasonable defaults rather than optimized via cross-validation, in order to prioritize methodological correctness over marginal performance gains.
7. **Refit cadences chosen for plausibility, not optimality.** The 21-/63-trading-day refit schedules approximate realistic operational cadences but were not tuned against any performance metric.

Chapter 9

Conclusion

This project implemented and fairly compared a classical statistical model, a classical machine-learning model, and a deep-learning model on an identical one-day-ahead volatility forecasting task, under a walk-forward backtesting protocol with explicit leakage checks. The central finding is that the model ranking depends on which regime and which loss function is examined: the LSTM wins on average, but GARCH(1,1) wins specifically during high-volatility periods and on the QLIKE scoring rule most sensitive to those periods, a genuinely decision-relevant result rather than a single leaderboard number.

Directions for extending this work include GJR-GARCH or EGARCH specifications to capture the leverage effect and Student- t innovations to address fat tails; walk-forward retraining of the LSTM, matching the treatment already given to GARCH and XGBoost; a market-implied-volatility baseline (the VIX index) to test whether any of these models add information beyond what the options market already prices in; an options-trading overlay tied to Hull's option-pricing chapters, using the volatility forecast to size a short-dated straddle position and evaluating realized P&L net of transaction costs; and intraday realized-volatility construction to obtain a materially less noisy evaluation target.

Appendix A

Full Implementation Code

The following is the complete Python implementation used to produce every result in this report, executed as a single Google Colab notebook.

```
1 # Install packages not preinstalled on Colab
2 !pip install -q yfinance arch
3
4
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8 from scipy import stats
9 import tensorflow as tf
10
11 pd.set_option("display.width", 120)
12
13
14 TICKER = "^GSPC"           # S&P 500 index. Try "^NSEI" for Nifty 50 if you prefer.
15 START  = "2012-01-01"
16 END    = "2026-07-01"
17
18 TEST_FRACTION      = 0.20   # chronological hold-out, most recent 20% of days
19 GARCH_REFIT_EVERY  = 21     # trading days (~monthly) - expanding window
20 XGB_REFIT_EVERY    = 63     # trading days (~quarterly) - expanding window
21 LSTM_SEQ_LEN       = 20     # days of history the LSTM looks back on
22 LSTM_EPOCHS        = 30
23 LSTM_N_SEEDS       = 5      # ensemble size -- see section 6 for why this matters
24
25
26 import yfinance as yf
27
28 raw = yf.download(TICKER, start=START, end=END, auto_adjust=True, progress=False)
29 raw = raw[["Close"]].copy()
30 raw.columns = ["Close"]
31 raw["log_ret"] = np.log(raw["Close"] / raw["Close"].shift(1))
32 raw = raw.dropna()
33
34 print(f"{len(raw)} trading days, {raw.index[0].date()} to {raw.index[-1].date()}")
35 raw.head()
36
37
38 fig, ax = plt.subplots(figsize=(10, 4))
39 ax.plot(raw.index, raw["Close"])
40 ax.set_title(f"{TICKER} closing price")
41 plt.show()
42
43
```

```

44 def build_dataset(df, ewma_lambda=0.94):
45     out = df.copy()
46     r = out["log_ret"]
47     r_lag1 = r.shift(1) # r[t-1]: the most recent return known at prediction time
48
49     out["target_var"] = r ** 2
50     out["target_vol"] = np.sqrt(out["target_var"] * 252)
51
52     for lag in range(1, 6):
53         out[f"ret_lag{lag}"] = r.shift(lag)
54         out[f"sqret_lag{lag}"] = r.shift(lag) ** 2
55
56     for w in (5, 10, 21):
57         out[f"roll_std_{w}"] = r_lag1.rolling(w).std()
58         out[f"roll_absmean_{w}"] = r_lag1.abs().rolling(w).mean()
59
60     # RiskMetrics-style EWMA volatility, built from r[t-1] backwards
61     rs = r_lag1.fillna(0.0).values
62     ewma_var = np.zeros(len(r))
63     ewma_var[0] = rs[0] ** 2
64     for t in range(1, len(r)):
65         ewma_var[t] = ewma_lambda * ewma_var[t - 1] + (1 - ewma_lambda) * rs[t] ** 2
66     out["ewma_vol"] = np.sqrt(ewma_var * 252)
67
68     return out.dropna()
69
70
71 FEATURE_COLS = (
72     [f"ret_lag{i}" for i in range(1, 6)]
73     + [f"sqret_lag{i}" for i in range(1, 6)]
74     + [f"roll_std_{w}" for w in (5, 10, 21)]
75     + [f"roll_absmean_{w}" for w in (5, 10, 21)]
76     + ["ewma_vol"]
77 )
78
79 ds = build_dataset(raw)
80 print(ds.shape)
81 ds[["target_vol", "ewma_vol"] + FEATURE_COLS[:3]].head()
82
83
84 def chrono_split(df, test_frac=0.2):
85     n_test = int(len(df) * test_frac)
86     return df.iloc[:-n_test].copy(), df.iloc[-n_test:].copy()
87
88 train, test = chrono_split(ds, TEST_FRACTION)
89 print(f"train={len(train)} rows ({train.index[0].date()} to {train.index[-1].date()})")
90 print(f"test = {len(test)} rows ({test.index[0].date()} to {test.index[-1].date()})")
91
92 fig, ax = plt.subplots(figsize=(10, 4))
93 ax.plot(ds.index, ds["target_vol"], linewidth=0.6)
94 ax.axvline(test.index[0], color="red", linestyle="--", label="train / test split")
95 ax.set_title("Realized volatility proxy (annualized)")
96 ax.legend()
97 plt.show()
98
99
100 from arch import arch_model
101
102 def garch_walk_forward(full_df, train, test, refit_every=21):

```

```

103     returns_pct = full_df["log_ret"] * 100
104     test_idx = test.index
105     forecasts = pd.Series(index=test_idx, dtype=float)
106     fitted_params = None
107
108     for i, t in enumerate(test_idx):
109         loc = full_df.index.get_loc(t)
110         history = returns_pct.iloc[:loc]
111
112         if fitted_params is None or i % refit_every == 0:
113             am = arch_model(history, vol="Garch", p=1, q=1, dist="normal", rescale=False)
114             res = am.fit(dispatch="off")
115             fitted_params = res.params
116             fc = res.forecast(horizon=1, reindex=False)
117         else:
118             am = arch_model(history, vol="Garch", p=1, q=1, dist="normal", rescale=False)
119             res_fixed = am.fix(fitted_params)
120             fc = res_fixed.forecast(horizon=1, reindex=False)
121
122         var_pct2 = fc.variance.values[-1, 0]
123         daily_var = var_pct2 / (100 ** 2)
124         forecasts.loc[t] = np.sqrt(daily_var * 252)
125
126     return forecasts
127
128 garch_fc = garch_walk_forward(ds, train, test, refit_every=GARCH_REFIT EVERY)
129 print("GARCH walk-forward done.")
130 garch_fc.head()
131
132
133 import xgboost as xgb
134
135 def xgb_walk_forward(train, test, refit_every=63):
136     test_idx = test.index
137     forecasts = pd.Series(index=test_idx, dtype=float)
138     combined = pd.concat([train, test])
139     train_end_pos = len(train)
140     model = None
141
142     for i, t in enumerate(test_idx):
143         cur_pos = train_end_pos + i
144         if model is None or i % refit_every == 0:
145             hist = combined.iloc[:cur_pos]
146             model = xgb.XGBRegressor(
147                 n_estimators=300, max_depth=3, learning_rate=0.05,
148                 subsample=0.8, colsample_bytree=0.8, random_state=0,
149             )
150             model.fit(hist[FEATURE_COLS], np.sqrt(hist["target_var"] * 252))
151
152         row = combined.iloc[[cur_pos]][FEATURE_COLS]
153         forecasts.loc[t] = model.predict(row)[0]
154
155     return forecasts
156
157 xgb_fc = xgb_walk_forward(train, test, refit_every=XGB_REFIT EVERY)
158 print("XGBoost walk-forward done.")
159 xgb_fc.head()
160
161

```

```

162 from sklearn.preprocessing import StandardScaler
163
164 def make_sequences(feature_arr, target_arr, seq_len):
165     X, y = [], []
166     for i in range(seq_len, len(feature_arr)):
167         X.append(feature_arr[i - seq_len:i])
168         y.append(target_arr[i])
169     return np.array(X), np.array(y)
170
171 def lstm_forecast(train, test, seq_len=20, epochs=30, seed=0):
172     tf.keras.utils.set_random_seed(seed)
173     scaler = StandardScaler().fit(train[FEATURE_COLS])
174     combined = pd.concat([train, test])
175     feat_scaled = scaler.transform(combined[FEATURE_COLS])
176     target_vol = np.sqrt(combined["target_var"].values * 252)
177
178     X_all, y_all = make_sequences(feat_scaled, target_vol, seq_len)
179     n_train_seq = len(train) - seq_len
180     X_train, y_train = X_all[:n_train_seq], y_all[:n_train_seq]
181     X_test = X_all[n_train_seq:]
182
183     model = tf.keras.Sequential([
184         tf.keras.layers.Input(shape=(seq_len, X_all.shape[2])),
185         tf.keras.layers.LSTM(32, activation="tanh"),
186         tf.keras.layers.Dense(16, activation="relu"),
187         tf.keras.layers.Dense(1, activation="softplus"),
188     ])
189     model.compile(optimizer="adam", loss="mse")
190     model.fit(X_train, y_train, epochs=epochs, batch_size=32, verbose=0,
191             validation_split=0.1,
192             callbacks=[tf.keras.callbacks.EarlyStopping(patience=5, restore_best_weights=
193                 True)])
194
195     preds = model.predict(X_test, verbose=0).flatten()
196     return pd.Series(preds, index=test.index)
197
198 def lstm_forecast_ensemble(train, test, seq_len=20, epochs=30, n_seeds=5):
199     per_seed = []
200     for seed in range(n_seeds):
201         fc = lstm_forecast(train, test, seq_len=seq_len, epochs=epochs, seed=seed)
202         per_seed.append(fc.rename(f"seed_{seed}"))
203     stacked = pd.concat(per_seed, axis=1)
204     return stacked.mean(axis=1), stacked.std(axis=1), stacked
205
206 lstm_fc, lstm_std, lstm_stack = lstm_forecast_ensemble(
207     train, test, seq_len=LSTM_SEQ_LEN, epochs=LSTM_EPOCHS, n_seeds=LSTM_N_SEEDS)
208 print(f"LSTM ensemble of {LSTM_N_SEEDS} seeds done. "
209       f"Mean seed-to-seed std across test days: {lstm_std.mean():.4f} "
210       f"(compare to the typical vol level of ~{test['target_vol'].mean():.2f} to judge
211         materiality)")
212
213 lstm_fc.head()
214
215 def qlike(y_true_var, y_pred_var):
216     ratio = y_true_var / y_pred_var
217     return np.mean(ratio - np.log(ratio) - 1)
218
219 def evaluate(name, y_true_vol, y_pred_vol):
220     y_true_vol = np.asarray(y_true_vol)

```

```

219     y_pred_vol = np.clip(np.asarray(y_pred_vol), 1e-4, None)
220     y_true_var, y_pred_var = (y_true_vol ** 2) / 252, (y_pred_vol ** 2) / 252
221     rmse = np.sqrt(np.mean((y_true_vol - y_pred_vol) ** 2))
222     mae = np.mean(np.abs(y_true_vol - y_pred_vol))
223     ql = qlike(np.clip(y_true_var, 1e-12, None), np.clip(y_pred_var, 1e-12, None))
224     ss_res = np.sum((y_true_vol - y_pred_vol) ** 2)
225     ss_tot = np.sum((y_true_vol - np.mean(y_true_vol)) ** 2)
226     r2 = 1 - ss_res / ss_tot
227     return {"model": name, "RMSE": rmse, "MAE": mae, "QLIKE": ql, "R2": r2}
228
229 def diebold_mariano(err1, err2):
230     d = np.asarray(err1) ** 2 - np.asarray(err2) ** 2
231     n = len(d)
232     dm_stat = d.mean() / np.sqrt(d.var(ddof=0) / n)
233     p_value = 2 * (1 - stats.norm.cdf(np.abs(dm_stat)))
234     return dm_stat, p_value
235
236 true_vol = test["target_vol"]
237 forecasts = {"GARCH(1,1)": garch_fc, "XGBoost": xgb_fc, "LSTM": lstm_fc}
238
239 results_df = pd.DataFrame([evaluate(name, true_vol, fc) for name, fc in forecasts.items()])
240 print(results_df)
241
242 dm_table = []
243 names = list(forecasts.keys())
244 for i in range(len(names)):
245     for j in range(i + 1, len(names)):
246         a, b = names[i], names[j]
247         stat, p = diebold_mariano(true_vol - forecasts[a], true_vol - forecasts[b])
248         dm_table.append((a, b, stat, p))
249         sig = "significant at 5%" if p < 0.05 else "not significant at 5%"
250         print(f"{a} vs {b}: DM={stat:.3f}, p={p:.3f} ({sig})")
251
252
253 fig, axes = plt.subplots(3, 1, figsize=(10, 10), sharex=True)
254 for ax, (name, fc) in zip(axes, forecasts.items()):
255     ax.plot(true_vol.index, true_vol.values, label="Realized (actual)", linewidth=0.7,
256            alpha=0.7)
257     ax.plot(fc.index, fc.values, label=f"{name} forecast", linewidth=1.0)
258     ax.set_title(name)
259     ax.legend()
259 plt.tight_layout()
260 plt.show()
261
262
263 def regime_breakdown(true_vol, forecasts, n_bins=3):
264     labels = ["Low", "Medium", "High"][:n_bins]
265     regime = pd.qcut(true_vol, n_bins, labels=labels)
266     rows = []
267     for name, fc in forecasts.items():
268         for lbl in labels:
269             mask = (regime == lbl).values
270             if mask.sum() < 5:
271                 continue
272             m = evaluate(name, true_vol.values[mask], fc.values[mask])
273             m["regime"] = lbl
274             m["n_days"] = int(mask.sum())
275             rows.append(m)
276     return pd.DataFrame(rows)

```

```

277
278 def spike_capture(true_vol, forecasts, top_pct=0.05):
279     n_top = max(1, int(len(true_vol) * top_pct))
280     top_idx = true_vol.sort_values(ascending=False).index[:n_top]
281     rows = []
282     for name, fc in forecasts.items():
283         ratio = (fc.loc[top_idx] / true_vol.loc[top_idx]).mean()
284         rows.append({"model": name, "n_spike_days": n_top, "avg_pred_over_actual": ratio})
285     return pd.DataFrame(rows)
286
287 regime_df = regime_breakdown(true_vol, forecasts, n_bins=3)
288 spike_df = spike_capture(true_vol, forecasts, top_pct=0.05)
289
290 print(regime_df[["model", "regime", "n_days", "RMSE", "QLIKE"]])
291 print()
292 print(spike_df)
293
294
295 from matplotlib.backends.backend_pdf import PdfPages
296
297 def build_report(pdf_path, meta, raw_df, ds, train, test, forecasts, results_df, dm_table,
298                 regime_df=None, spike_df=None, lstm_stability=None):
299     true_vol = test["target_vol"]
300
301     def text_page(pdf, title, lines, fontsize=11, line_spacing=0.035, start_y=0.88,
302                  bottom_margin=0.05):
303         # Auto-paginate: a fixed-height page can only fit so many lines, and
304         # silently overflowing past the bottom margin clips content rather
305         # than raising an error, so it's worth guarding against explicitly.
306         max_lines = max(1, int((start_y - bottom_margin) / line_spacing))
307         chunks = [lines[i:i + max_lines] for i in range(0, len(lines), max_lines)] or [[]]
308         for idx, chunk in enumerate(chunks):
309             page_title = title if idx == 0 else f"{title} (cont'd)"
310             fig = plt.figure(figsize=(8.27, 11.69))
311             fig.text(0.08, 0.94, page_title, fontsize=16, fontweight="bold")
312             y = start_y
313             for line in chunk:
314                 fig.text(0.08, y, line, fontsize=fontsize, wrap=True, va="top")
315                 y -= line_spacing
316             plt.axis("off")
317             pdf.savefig(fig)
318             plt.close(fig)
319
320     with PdfPages(pdf_path) as pdf:
321         text_page(pdf, "Volatility Forecasting Shootout", [
322             "GARCH(1,1) vs. XGBoost vs. LSTM", "",
323             f"Instrument: {meta['ticker']}",
324             f"Sample period: {meta['start']} to {meta['end']}",
325             f"Total observations (after feature construction): {len(ds)}",
326             f"Train / Test split: {len(train)} / {len(test)} (chronological, no shuffling)",
327             "",
328             "Target definition:",
329             "    target_var[t] = r[t]^2 (realized-variance proxy for day t)",
330             "    target_vol[t] = sqrt(target_var[t] * 252) (annualized)",
331             "Every feature at row t uses ONLY r[t-1], r[t-2], ... -- never r[t] itself --",
332             "so all three models forecast the same one-day-ahead quantity from the same",
333             "information set. An off-by-one error here is a classic source of inflated,",
334             "non-reproducible backtest results.",
335             f"GARCH refit cadence: every {meta['refit_every_garch']} trading days (

```

```

    expanding window)",
334     f"XGBoost refit cadence: every {meta['refit_every_xgb']} trading days (
        expanding window)",
335     f"LSTM: {meta.get('n_seeds', 1)}-seed ensemble average, sequence length {meta['
        seq_len']} days", "",
336     "Evaluation metrics: RMSE and MAE (vol units), QLIKE (variance units -- the",
337     "standard scoring rule for volatility forecasts), and the Diebold-Mariano test"
    ,
338     "for whether accuracy differences are statistically significant.",
339 ]))
340
341 fig, axes = plt.subplots(2, 1, figsize=(8.27, 11.69))
342 axes[0].plot(raw_df.index, raw_df["Close"])
343 axes[0].set_title(f"{meta['ticker']} price level")
344 axes[1].plot(ds.index, ds["target_vol"], linewidth=0.6)
345 axes[1].set_title("Realized volatility proxy (annualized)")
346 axes[1].axvline(test.index[0], color="red", linestyle="--", linewidth=1, label="
    train/test split")
347 axes[1].legend()
348 fig.tight_layout()
349 pdf.savefig(fig)
350 plt.close(fig)
351
352 for name, fc in forecasts.items():
353     fig, ax = plt.subplots(figsize=(8.27, 5))
354     ax.plot(true_vol.index, true_vol.values, label="Realized (actual)", linewidth
        =0.8, alpha=0.7)
355     ax.plot(fc.index, fc.values, label=f"{name} forecast", linewidth=1.0)
356     ax.set_title(f"{name}: forecast vs. realized volatility (test period)")
357     ax.legend()
358     fig.tight_layout()
359     pdf.savefig(fig)
360     plt.close(fig)
361
362 fig, axes = plt.subplots(1, 3, figsize=(11, 4))
363 for ax, metric in zip(axes, ["RMSE", "MAE", "QLIKE"]):
364     ax.bar(results_df["model"], results_df[metric], color=["#4C72B0", "#DD8452", "
        #55A868"])
365     ax.set_title(metric)
366     ax.tick_params(axis="x", rotation=20)
367 fig.suptitle("Model comparison (lower is better on all three)")
368 fig.tight_layout()
369 pdf.savefig(fig)
370 plt.close(fig)
371
372 # Calibration: predicted vs actual, points below the diagonal = under-prediction
373 fig, axes = plt.subplots(1, 3, figsize=(13, 4.5))
374 for ax, (name, fc) in zip(axes, forecasts.items()):
375     ax.scatter(true_vol.values, fc.values, s=6, alpha=0.35)
376     lim = max(true_vol.max(), fc.max())
377     ax.plot([0, lim], [0, lim], "r--", linewidth=1, label="perfect calibration")
378     ax.set_xlabel("Actual realized vol")
379     ax.set_ylabel("Predicted vol")
380     ax.set_title(name)
381     ax.legend(fontsize=8)
382 fig.suptitle("Calibration: points below the red line are under-predictions")
383 fig.tight_layout()
384 pdf.savefig(fig)
385 plt.close(fig)

```

```

386
387     if regime_df is not None and spike_df is not None:
388         lines = [
389             "A single pooled RMSE/QLIKE number can hide a model that performs well",
390             "on ordinary days but quietly fails exactly when volatility spikes --",
391             "which is the scenario that actually matters for risk management and",
392             "options trading. So the test period is split into Low / Medium / High",
393             "realized-volatility terciles and every model is re-scored within each.", "
394             ",
395             f"{'Model':<13}{ 'Regime':<9}{ 'N':>5}{ 'RMSE':>10}{ 'QLIKE':>10}",
396         ]
397         for _, row in regime_df.iterrows():
398             lines.append(f"{'row['model']':<13}{row['regime']:<9}{row['n_days']:>5}{row['
399                 RMSE']:>10.4f}{row['QLIKE']:>10.4f}")
400             high = regime_df[regime_df["regime"] == "High"].sort_values("QLIKE")
401             lines.append("")
402             if len(high):
403                 lines.append(f"Best QLIKE in the High-volatility regime: {high.iloc[0]['
404                     model']}")
405             lines += [f" ", f"Spike capture -- average (predicted / actual) on the worst "
406                 f"{'spike_df.iloc[0]['n_spike_days']}' test days by realized vol:",
407                 f"{'Model':<13}{ 'pred / actual':>15}"]
408             for _, row in spike_df.iterrows():
409                 flag = " <-- under-predicts spikes" if row["avg_pred_over_actual"] < 0.85
410                 else ""
411                 lines.append(f"{'row['model']':<13}{row['avg_pred_over_actual']:>15.3f}{flag}
412                     ")
413             lines += [f" ", "A ratio well below 1.0 means the model systematically under-
414                 calls",
415                 "volatility on exactly the days where under-calling it is costliest
416                 --",
417                 "e.g. an under-hedged options book right before a vol spike."]
418             text_page(pdf, "Regime Breakdown & Tail-Spike Capture", lines, fontsize=10)
419
420         lines = ["Metric table (pooled, whole test period):", "",
421             f"{'Model':<14}{ 'RMSE':>10}{ 'MAE':>10}{ 'QLIKE':>10}{ 'R2':>10}"]
422         for _, row in results_df.iterrows():
423             lines.append(f"{'row['model']':<14}{row['RMSE']:>10.4f}{row['MAE']:>10.4f}{row['
424                 QLIKE']:>10.4f}{row['R2']:>10.4f}")
425         lines += [f" ", "Diebold-Mariano tests (squared-error loss differential):"]
426         for a, b, stat, p in dm_table:
427             sig = "significant at 5%" if p < 0.05 else "not significant at 5%"
428             lines.append(f" {a} vs {b}: DM={stat:.3f}, p={p:.3f} ({sig})")
429
430         best_rmse = results_df.sort_values("RMSE").iloc[0]["model"]
431         best_qlike = results_df.sort_values("QLIKE").iloc[0]["model"]
432         lines += [f" ", f"Lowest pooled RMSE: {best_rmse}", f"Lowest pooled QLIKE: {
433             best_qlike}"]
434         if lstm_stability is not None:
435             lines += [f" ", f"LSTM ensemble stability: mean seed-to-seed std = {
436                 lstm_stability:.4f}",
437                 " (GARCH and XGBoost are deterministic given the data; this is the",
438                 " price of the LSTM's flexibility -- controlled here by averaging
439                 seeds)"]
440         lines += [f" ", "Recommendation:"]
441         if best_rmse != best_qlike:
442             lines += [f" RMSE and QLIKE disagree on the winner ({best_rmse} vs {best_qlike
443                 }).",
444                 " This is a real, informative disagreement, not noise: RMSE is

```



```

433         dominated",
434         " by the many ordinary days, while QLIKE weights large-variance days
           far",
435         " more heavily. See the regime/spike-capture page for which model
           actually",
436         " degrades during volatility spikes -- that model is the riskier
           deploy",
437         " choice even if it wins on pooled RMSE."],
438     else:
439         lines += [f" {best_rmse} wins on both pooled RMSE and QLIKE -- check the
           regime",
440         " breakdown to confirm that holds specifically in the High-vol
           regime too."]
441     lines += ["", " In practice: GARCH is the cheapest to retrain and the most
           interpretable",
442         " -- a reasonable production default or sanity-check floor. An ML/DL
           model",
443         " should only replace it if it demonstrably wins where GARCH is weakest"
           ,
444         " WITHOUT losing GARCH's responsiveness to fresh shocks -- which is
           exactly",
445         " what the regime table above checks."]
446     text_page(pdf, "Results & Discussion", lines, fontsize=10)
447     return pdf_path
448
449 meta = dict(ticker=TICKER, start=START, end=END,
450             refit_every_garch=GARCH_REFIT_EVERY, refit_every_xgb=XGB_REFIT_EVERY,
451             seq_len=LSTM_SEQ_LEN, n_seeds=LSTM_N_SEEDS)
452
453 pdf_path = build_report("quantbox_volatility_report.pdf", meta, raw, ds, train, test,
454                        forecasts, results_df, dm_table, regime_df=regime_df, spike_df=
455                        spike_df,
456                        lstm_stability=float(lstm_std.mean()))
457 print("Saved:", pdf_path)
458
459 # Download the PDF (Colab only -- comment this out if running locally)
460 try:
461     from google.colab import files
462     files.download(pdf_path)
463 except ImportError:
464     print("Not running in Colab -- find the file at:", pdf_path)

```

Listing A.1: Complete project implementation: data collection through PDF report generation.